

Republic of Cameroon

Peace-Work-Fatherland

Ministry of Higher Education

University of Buea

Faculty of Engineering and
Technology

Department of Computer
Engineering



République du Cameroun

Paix-Travail-Patrie

Ministère de l'enseignement
supérieures

Université de Buea

Faculty of Engineering and
Technology

Department of Computer
Engineering

CAR FAULT DIAGNOSTIC APP: UI DESIGN AND FRONTEND IMPLEMENTATION

Course Instructor : Dr Nkemeni Valery

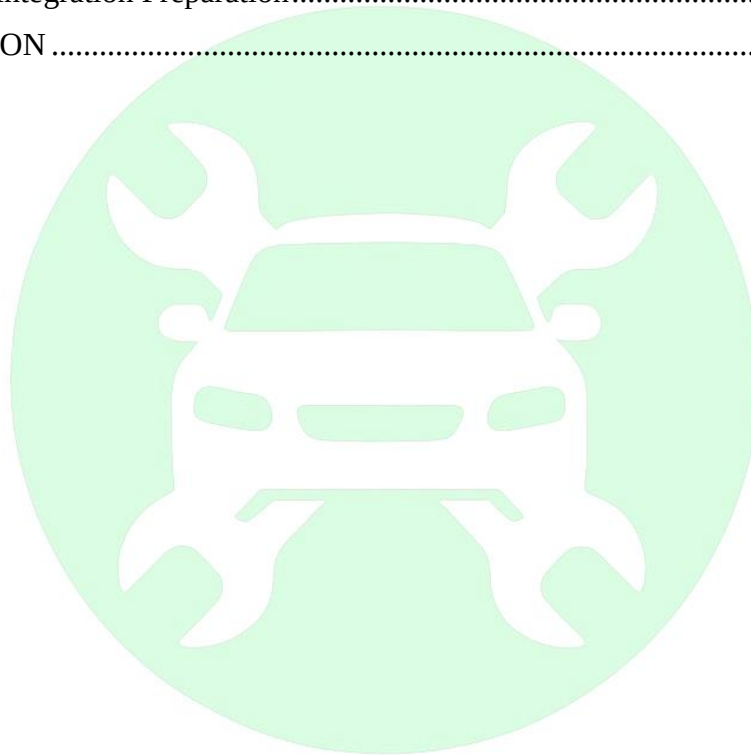
Course code : CEF440

Group Members

- | | |
|--|-----------------|
| 1- Angi Atangwa Valerine | FE22A148 |
| 2- Av'Nyahbeuhkonh Ezekiel Hosana | FE22A160 |
| 3- Beng Dwe Meinoff | FE22A173 |
| 4- Jong Turkson Junior | FE22A228 |
| 5- Nyuyesemo Calglain | FE22A287 |

Table of Contents

ABSTRACT	2
I- INTRODUCTION.....	3
II- APP'S IDENTITY	3
III- USER INTERFACE DESIGN	6
IV- FRONTEND IMPLEMENTATION	8
IV.1 Technology Used	8
Why React Native Instead of Flutter?	9
IV.2- Structure of the Application.....	10
IV.3 UI Consistency with Design	10
IV.4 Backend Integration Preparation.....	12
V- CONCLUSION	13



ABSTRACT

This report outlines the user interface (UI) design and frontend implementation of the **MFix mobile application**, a smart diagnostic platform for car owners and mechanics. The purpose of the app is to enable users to detect car faults using image and sound data, and to connect with nearby service providers. The app was designed using **React Native**, ensuring cross-platform support and scalability. Key UI features include a modern green-themed interface, role-based sign-up, car status displays, diagnostic tools, and a live map to locate nearby mechanics. This report details the app's identity, visual design choices, and the frontend development strategies employed, including readiness for backend integration, error handling, and content management.



I- INTRODUCTION

The MFix application aims to bridge the gap between car users and automotive repair services through a mobile-based diagnostic platform. As the demand for smart, real-time solutions in the automotive sector increases, MFix serves as an intuitive interface for detecting faults and providing immediate actionable support. The UI plays a critical role in facilitating user interactions by presenting a clean, functional, and consistent visual experience.

This report provides insight into how the UI was designed and implemented. We examine the app's identity, visual styling, and technical structure to ensure the interface meets user expectations while remaining flexible for future enhancements.

II- APP'S IDENTITY

App Identity here refers to the distinct personality and branding of our mobile application. It includes the app's **name, logo, colour theme, target users, and scope**. A strong app identity helps users immediately recognize the app, trust its functionality, and remember it over time. It communicates what the app does and why it's valuable. Our app identity ensures:

- **Consistency** across all screens and platforms.
- **Clarity** in purpose and usability.
- **Emotional connection** through visual and functional appeal.

Our Car Fault Diagnostic App has for name “MFix” where ‘M’ here stands for Moto which refers to car in Cameroon native language, pidgin.

Why MFix ? : MFix was chosen because it reflects the main purpose of the app which is diagnosing vehicle issues using a mobile device.

Also, MFix is easy to pronounce and remember, even for non-technical users, it is also suitable for creating logos, splash screens, and marketing materials hence making it branding friendly.

Scope of MFix

Scope:

MFix is a mobile app that provides real-time monitoring and fault detection capabilities by scanning vehicle's dashboard and recognising engine sounds. It interprets diagnostic trouble, displays readable fault descriptions, suggests possible causes, and recommends solutions or next steps. The app aims to empower users with actionable insights without requiring advanced automotive knowledge. Features include fault history logs, live engine parameter viewing, maintenance reminders, and a user-friendly dashboard.

Core features

- Dashboard light/sign recognition via camera.
- Engine sound analysis using ML models.

- Multilingual support (English, French, local languages).
- Maintenance logs and reminders.
- Explanation of warnings, repair tutorials, and maintenance reminders.
- Geolocation-based mechanic/garage suggestions.
- Offline functionality for critical tasks.

MFix Target Users

The main targeted users of MFix are **car owners**. We also have others such as car users (drivers and mechanics)

MFix Colour Theme

The colour scheme of the MFix app consists of **green, black (background), and white**. This combination was carefully chosen for both **aesthetic** and **functional** reasons:

1. Green – *Innovation, Trust, Diagnostics*

- Green is associated with **technology, growth, safety, and diagnostic health**, which are central to MFix’s role as a smart vehicle diagnostic tool.
- It gives users a **sense of success and reliability**, especially when diagnostics return healthy results (e.g., a green “Normal” or “Charged” status).
- In UX, green is often used for actions like “Go”, “Success”, or “Good Condition” — exactly the kind of reassurance we want to give users.

2. Black (Dark Background) – *Modern, Clean, Battery-Friendly*

- A dark background provides **visual contrast** that makes colored elements like green and white **stand out more clearly**.
- It gives the app a **sleek and modern** look, commonly used in professional or tech-oriented tools.
- On mobile devices, dark themes are also **easier on the eyes** and **save battery life**, especially for OLED screens.

3. White – *Clarity and readability*

- White is used primarily for **text** and **icons**, ensuring **high readability** against the dark background.
- It represents **neutral information**, clear instructions, and keeps the interface clean and simple.

MFix Logo

The logo was designed to perfectly fit MFix purpose while having an appealing visual design maintaining our color themes

MFix logo features a modern **white car icon flanked by two crossed wrenches**, all set against a solid **green circular background**. This design was chosen to communicate the app's core purpose clearly and instantly:

Logo Symbolism

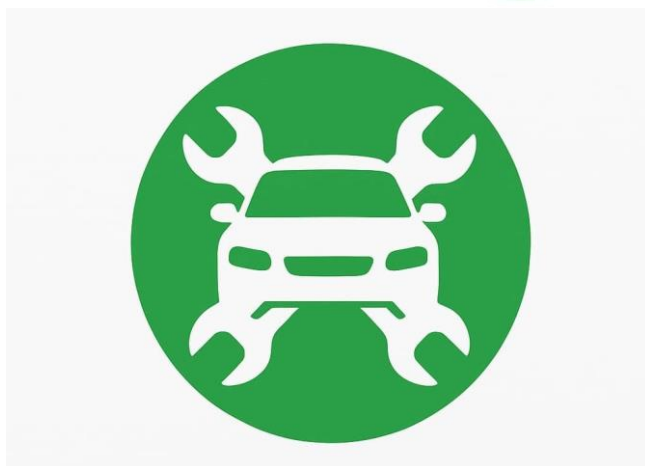
- **Car silhouette:** Represents the primary subject of the app – vehicles.
- **Wrenches:** A universal symbol for diagnostics, repair, and maintenance.
- **Green circle:** Aligns with our core color palette, symbolizing safety, reliability, and technology.

Why this logo works

- **Simple and recognizable:** The minimalistic design makes the logo easily identifiable even at small sizes.
- **Professional yet friendly:** The use of soft curves and tools reflects trustworthiness without being too technical or intimidating for everyday car owners.
- **Scalable:** The design maintains clarity whether used on splash screens, dashboards, or app icons.

Brand alignment

- The logo reinforces **MFix's identity** as a smart diagnostic solution.
- It visually complements the app's color scheme (green, black, white) and design style.
- This logo helps position MFix as both **user-friendly** and **technologically capable**, appealing to mechanics, car owners, and service centers alike.



MFIX LOGO

III- USER INTERFACE DESIGN

User interface of an app is the point of interaction that exists between the users and the application, MFix users interface follows the various design principles and different usability criteria that applies to our system.

The various design principles our system follows includes;

1. Consistency

All screens follow a unified theme of green, white, and black, using the same button styles, spacing, and iconography for familiarity and predictability.

2. Simplicity and Minimalism

Each screen contains only essential actions and information (e.g., Sign-Up form, Diagnosis results, Search map), avoiding unnecessary graphics or text.

3. Visibility and Feedback

Loading indicators, placeholder text, error alerts, and visual feedback (e.g., active tab highlights) make user interactions clear and responsive.

4. Accessibility

High contrast colors (green on black/white), large readable fonts, intuitive icons, and clear labels improve accessibility for all users.

5. User Control and Freedom

Users are able to easily go back or undo actions.

Navigation includes a back arrow and clean transitions, allowing users to switch tabs or cancel without confusion.

In order to implement the design principles outlined above, we used **Figma** as the primary tool for prototyping and UI design. **Figma** allowed us to:

- **Create a consistent layout** using reusable components and shared styles.
- **Ensure visual hierarchy** through size, spacing, and color contrast.
- **Simulate user flow** and navigation between screens.

- **Collaborate efficiently** and make real-time design updates.
- **Preview responsiveness** across various device screen sizes.

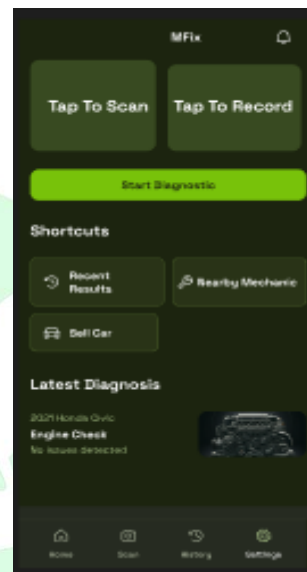
By applying the design principles directly in Figma, we ensured that each screen not only looked good but was also **usable, accessible, and functional**.

Below are some screen mockups designed using Figma that reflect our adherence to modern UI/UX principles:

1- Welcome Screen



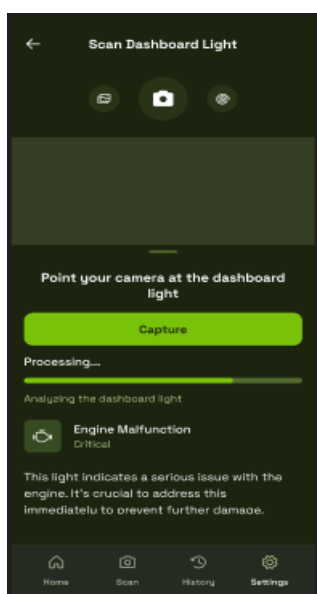
2- Home page



Clean layout showing app's name and slogan.

Display all app's major Characteristics

3- Scan dashboard



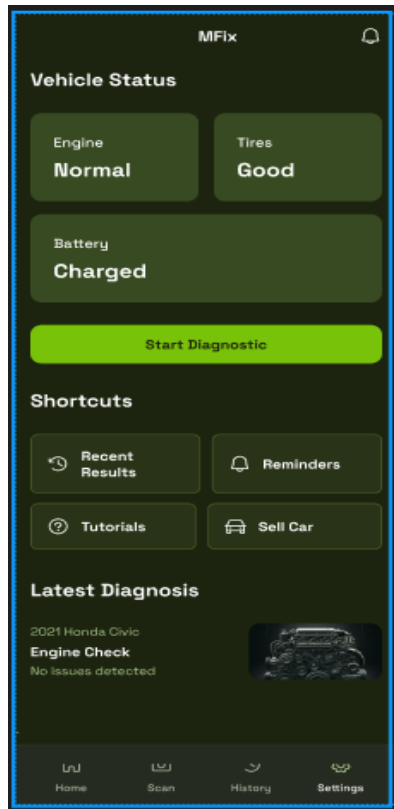
4- Record Sound



Enables user to scan dashboard

through camera

2- Vehicle status page



Permits user to view result after a scan or a sound record

IV- FRONTEND IMPLEMENTATION

In order to transform the MFix interface designs into a working mobile application, we developed the frontend using modern cross-platform tools. Our focus was to ensure that the app maintains a consistent visual identity, responds efficiently across devices, and remains fully prepared for backend integration. The frontend also includes measures for usability, responsiveness, and scalability in future updates.

IV.1 Technology Used

The development of the MFix mobile application involved a carefully selected technology stack to ensure performance, scalability, and maintainability:

- **JavaScript(ES6+):**
The primary programming language used for building the logic and interface of the

application. JavaScript's flexibility and wide ecosystem make it ideal for modern mobile app development.

- **React Native:**
A powerful framework built on JavaScript, used to develop cross-platform mobile applications for both Android and iOS from a single codebase. React Native enables native performance while retaining web-like development efficiency.
- **Expo:**
A development platform built around React Native that simplifies the build process, supports easy device previews, over-the-air updates, and provides access to native APIs without needing separate native code.
- **Figma:**
A UI/UX design tool used to create high-fidelity prototypes, wireframes, and screen layouts. Figma facilitated a collaborative and efficient design process aligned with our app's visual identity and design principles.
- **React Navigation:**
Used to handle routing and navigation across screens within the application, supporting stack, tab, and drawer navigation patterns.
- **React Native Maps:**
Integrated to display a real-time, interactive map interface, replacing static map images and enhancing user experience.
- **Context API:**
Utilized for global state management (e.g., user session, verification states), avoiding prop drilling and allowing centralized access to shared data.
- **AsyncStorage:**
A persistent storage system used to store lightweight data locally on the device, such as login tokens or user preferences.

Why React Native Instead of Flutter?

Initially, the MFix team planned to develop the mobile application using **Flutter** due to its high-performance rendering engine and elegant UI capabilities. However, during the early stages of development, we encountered significant hardware-related limitations that made it challenging to continue with Flutter.

Flutter requires a relatively **high-performance machine** to run smoothly, especially when building and running emulators or compiling large projects. The **development systems available to the team had limited hardware resources**, such as:

- **Low RAM capacity**
- **Slower processors**
- **Limited storage**
- **Incompatibility with Android Studio on some systems**

These issues led to **frequent build failures**, long compilation times, and poor performance of the development environment, which **hindered productivity and progress**.

On the other hand, **React Native**, especially when combined with **Expo**, proved to be significantly more **lightweight and developer-friendly**. It allowed us to:

- **Test and run on physical devices easily** via Expo Go
- Avoid heavy IDEs like Android Studio during the early phase
- Build and debug apps with less computational strain
- Leverage a browser-based simulator and hot reload efficiently

For these reasons, **hardware limitations** made it more practical and feasible to switch to **React Native**, ensuring smoother development without compromising the quality of the final product.

IV.2- Structure of the Application

The app is structured into reusable **screens**, **components**, and **hooks** to promote maintainability and scalability. Example screens include:

- **Authenticatio**
- **User**
- **Diagnostic**
- **Car**

Each screen is connected using **React Navigation** and leverages **useContext** for state management. This ensures user data like roles and sessions are not lost on navigation.

IV.3 UI Consistency with Design

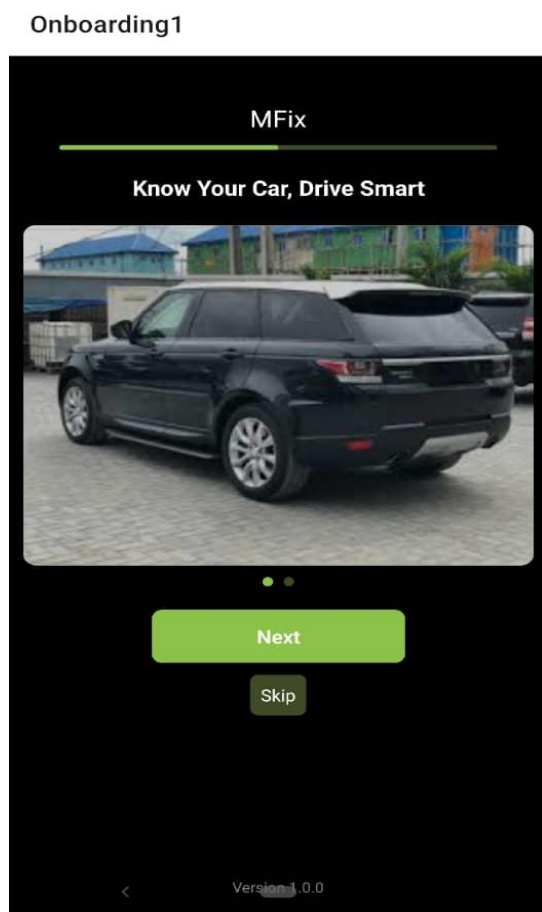
The React Native implementation **follows the Figma design though some slight changes were made such as the color of the bottom navbar**. Colors, fonts, layout spacing, and component hierarchy were reproduced using:

- StyleSheet for scoped styling
- ScrollView, View, TouchableOpacity, and other native components
- Consistent use of padding, margin, and font styles
- The interface uses `flexbox` layout and percentage-based widths (e.g. '48%', '100%') to ensure elements adapt across devices. Components are scrollable, and spacing is handled to prevent UI breakage on small screens.

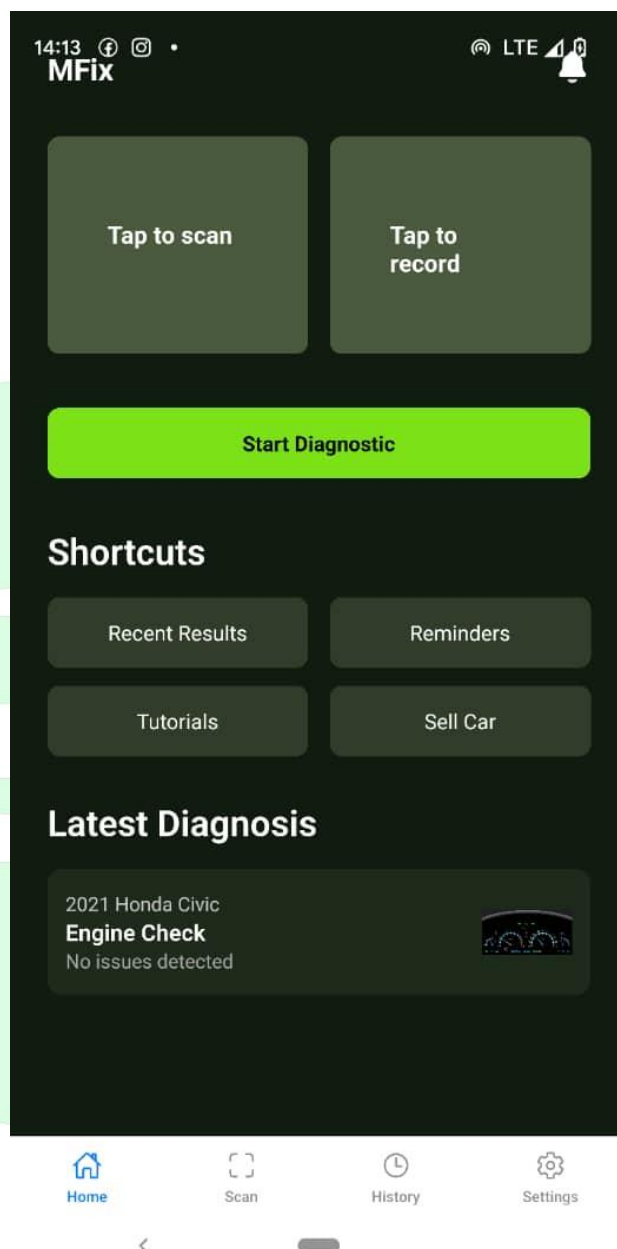
The color scheme — **black background**, **white text**, and **green highlights** — was preserved across all screens to ensure brand identity and readability.

Below are some of the pages of our app after frontend implementation

Welcome page



Home page



IV.4 Backend Integration Preparation

Although backend integration is pending, the UI is **ready for dynamic data**. We've:

- Created mock API functions using `setTimeout` and `useEffect` for testing lazy loading and error handling.
- Designed all content blocks (vehicle status, diagnosis result, etc.) to consume API responses later without any structural change.

The frontend of the MFix mobile application was carefully developed using React Native to ensure a responsive, intuitive, and visually consistent user interface. Each screen was implemented to match the UI design created in Figma, maintaining fidelity to color schemes, layouts, and user flow. The structure is modular and scalable, allowing for easy integration with backend APIs when available. With careful attention to content management, error handling, and lazy loading, the frontend stands ready for full functionality in a real-world environment.



V- CONCLUSION

This report presented the user interface design and implementation of the **MFix Car diagnostics application**, focusing on delivering a visually coherent, intuitive, and user-centered mobile experience. Starting with a strong app identity and consistent visual theme, we used Figma for prototyping and React Native for development to bring our design to life across various screens, including onboarding, diagnostics, and verification.

By adhering to fundamental design principles—such as consistency, clarity, and accessibility—and usability criteria like learnability and responsiveness, we ensured that the interface remains both functional and engaging. The implementation also incorporates key frontend practices including lazy loading, error handling, and content modularity, ensuring it is fully prepared for future backend integration. Moving forward, the system is well-positioned to support advanced features like real-time diagnostics, sound/image analysis, and cloud-based user data management, making it a strong foundation for a complete and efficient vehicle diagnostic solution.

